CUDA C/C++ BASICS

COOPERATING THREADS

CONCEPTS

- Heterogeneous Computing
- Blocks
- Threads
- Indexing
- Shared memory
- __syncthreads()
- Asynchronous operation
- Handling errors
- Managing devices

© NVIDIA 2013

Stencil

The word **stencil** means **a fixed pattern of neighboring elements** used to calculate a new value.

1D Stencil: Suppose we have a one-dimensional array

$$A = [A_0, A_1, A_2, \ldots, A_{N-1}]$$

Instead of processing each element alone, every new element depends on **its neighboring elements**.

$$B_i = f(A_{i-1}, A_i, A_{i+1})$$

Every output element is computed using its left neighbor, itself, and its right neighbor. This neighbor pattern is called a **stencil**.

Average

$$B_i = \frac{A_{i-1} + A_i + A_{i+1}}{3}$$

Weighted

$$B_i = 0.25A_{i-1} + 0.5A_i + 0.25A_{i+1}$$

Stencil

5-point stencil

$$B_i = \frac{A_{i-2} + A_{i-1} + A_i + A_{i+1} + A_{i+2}}{5}$$

Index	0	1	2	3	4	5	6
Value	5	10	15	20	25	30	35

$$B_1 = \frac{5 + 10 + 15}{3} = 10$$

$$B_2 = \frac{10 + 15 + 20}{3} = 15$$

$$B_3 = \frac{15 + 20 + 25}{3} = 20$$

$$B_4 = 25$$

$$B_5 = 30$$

Output: 10 15 20 25 30

Stencil

A stencil is useful when one value depends on nearby values.

Application	Purpose
Heat diffusion	Temperature propagation
Weather forecasting	Atmospheric simulation
Fluid dynamics	Flow of liquids and gases
Signal processing	Noise reduction
Image processing	Blur, sharpen, edge detection
Wave simulation	Sound and seismic modeling
Machine learning	Some convolution operations

1D Stencil

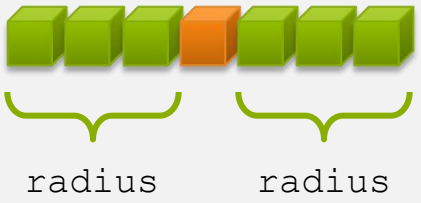
- Consider applying a 1D stencil to a 1D array of elements
  - Each output element is the sum of input elements within a radius

The radius tells us how many neighboring elements we look at on each side.

- If radius is 3, then each output element is the sum of 7 input elements:

Radius = 1
1 left + 1 current + 1 right = 3 elements

Number of elements used = 2 × radius + 1

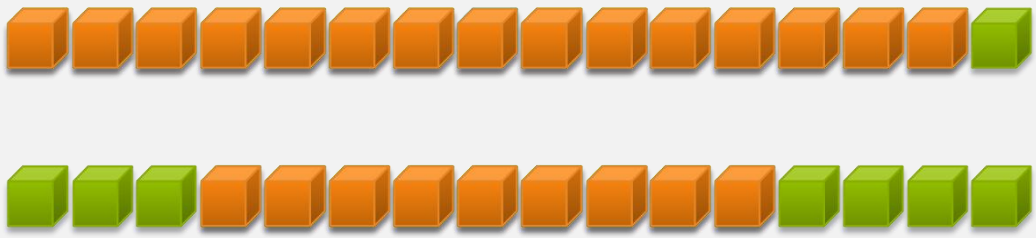


© NVIDIA 2013

Implementing Within a Block

- Each thread processes one output element
  - blockDim.x elements per block

- Input elements are read several times
  - With radius 3, each input element is read seven times



© NVIDIA 2013

Sharing Data Between Threads

- Within a block, threads share data via `shared memory`
- Extremely fast on-chip memory, user-managed
- Declare using `__shared__`, allocated per block
- Data is not visible to threads in other blocks

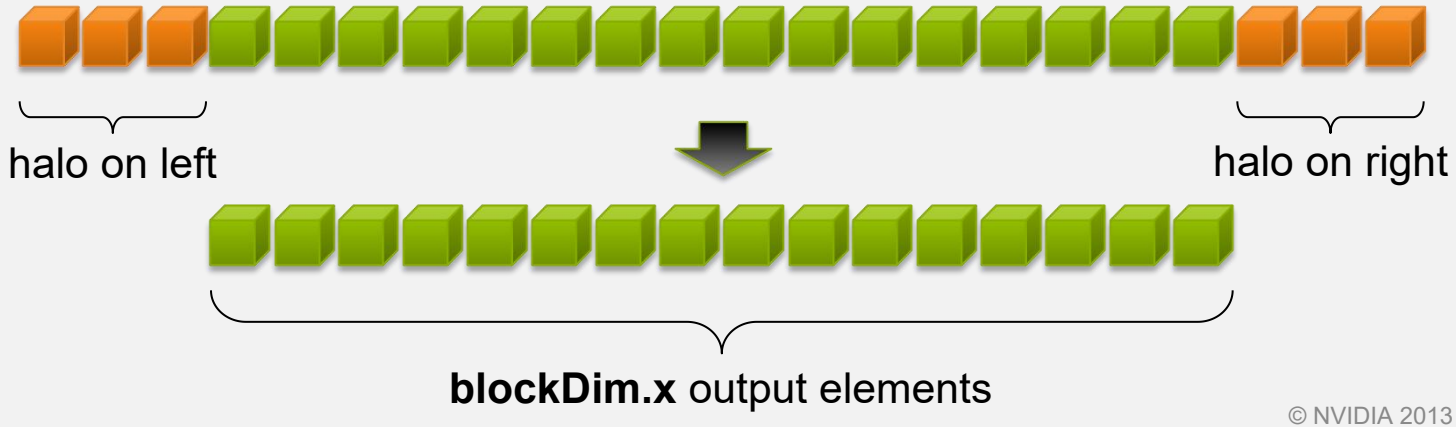
BLOCK_SIZE = 16
RADIUS = 3
We need space, 3 + 16 + 3 = 22

© NVIDIA 2013

Implementing With Shared Memory

- Cache data in shared memory
  - Read (**blockDim.x** + 2 * radius) input elements from global memory to shared memory
  - Compute **blockDim.x** output elements
  - Write **blockDim.x** output elements to global memory
  - Each block needs a halo (**ghost cells/region**, or **padding**) of radius elements at each boundary

A halo is the extra neighboring data around the main block data.



© NVIDIA 2013

Implementing With Shared Memory

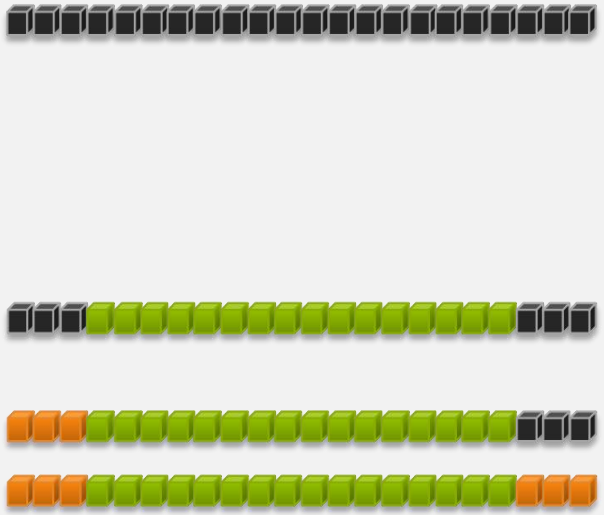
Code	Purpose
__shared__ int temp[...]	Creates fast shared memory for the block
gindex	Computes the element's position in global memory
lindex	Computes the corresponding position in shared memory, leaving room for halos
temp[lindex] = in[gindex]	Each thread copies its own element into shared memory
if (threadIdx.x < RADIUS)	Only the first **RADIUS** threads load halo values
temp[lindex - RADIUS]	Loads the left halo element
temp[lindex + BLOCK_SIZE]	Loads the right halo element
__syncthreads() (typically follows)	Ensures all shared-memory data is ready before stencil computation

"Everyone in this block, stop here and wait until everyone has reached this point."

Stencil Kernel

```
__global__ void stencil_1d(int *in, int *out) {
  __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
  int gindex = threadIdx.x + blockIdx.x * blockDim.x;
  int lindex = threadIdx.x + RADIUS;

  // Read input elements into shared memory
  temp[lindex] = in[gindex];
  if (threadIdx.x < RADIUS) {
    temp[lindex - RADIUS] = in[gindex - RADIUS];
    temp[lindex + BLOCK_SIZE] =
                        in[gindex + BLOCK_SIZE];
  }
```



BLOCK_SIZE = 16, RADIUS = 3

temp size = 16 + 2×3 = 22

Index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21
where
- 0-2 = left halo
- 3–18 = block data
- 19-21 = right halo

© NVIDIA 2013

Stencil Kernel

```
  // Apply the stencil
  int result = 0;
  for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
    result += temp[lindex + offset];

  // Store the result
  out[gindex] = result;
}
```

© NVIDIA 2013

Data Race!

- **The stencil will not work…**
- **Suppose thread 15 reads the halo before thread 0 has fetched it…**

```
temp[lindex] = in[gindex];
if (threadIdx.x < RADIUS) {
  temp[lindex – RADIUS = in[gindex – RADIUS];
  temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
}

int result = 0;
result += temp[lindex + 1];
```

Store at temp[18]



Skipped, threadIdx > RADIUS

Load from temp[19]



© NVIDIA 2013

__syncthreads()

- `void __syncthreads();`

- Synchronizes all threads within a block
  - Used to prevent RAW / WAR / WAW hazards

- All threads must reach the barrier
  - In conditional code, the condition must be uniform across the block

© NVIDIA 2013

Stencil Kernel

```
__global__ void stencil_1d(int *in, int *out) {
    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;
    int lindex = threadIdx.x + radius;

    // Read input elements into shared memory
    temp[lindex] = in[gindex];
    if (threadIdx.x < RADIUS) {
        temp[lindex - RADIUS] = in[gindex - RADIUS];
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
    }

    // Synchronize (ensure all the data is available)
    __syncthreads();
```

© NVIDIA 2013

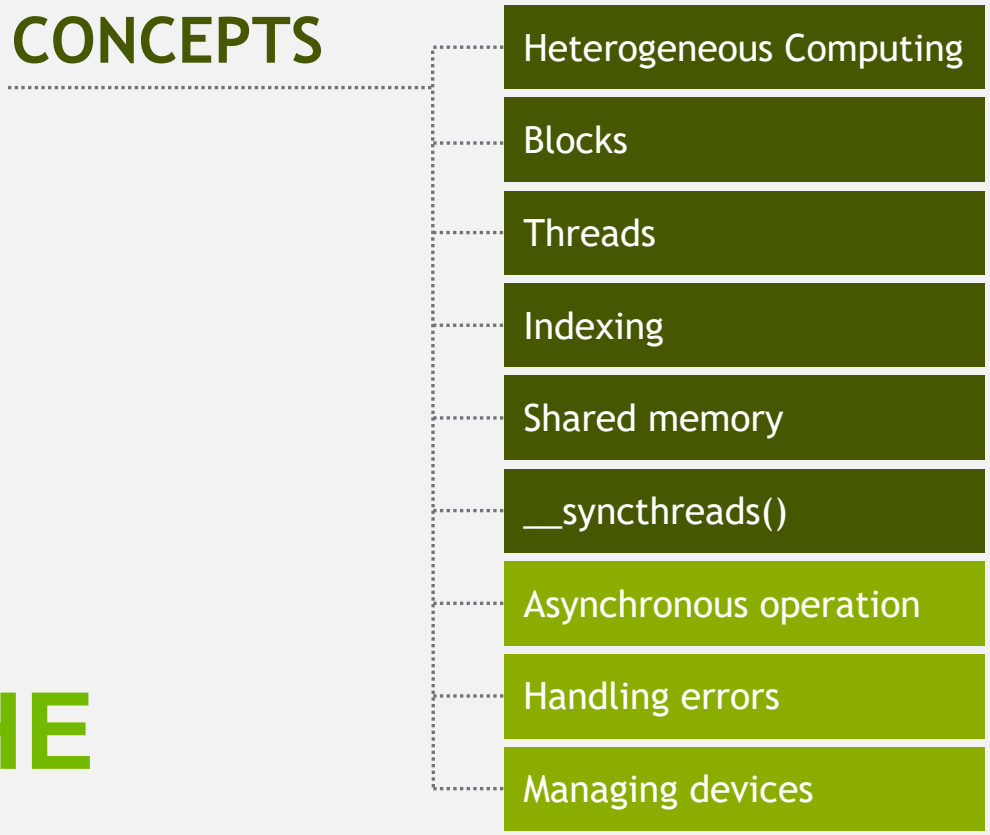
Stencil Kernel

```
    // Apply the stencil
    int result = 0;
    for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
        result += temp[lindex + offset];

    // Store the result
    out[gindex] = result;
}
```

© NVIDIA 2013

MANAGING THE DEVICE



© NVIDIA 2013

Coordinating Host & Device

- Kernel launches are asynchronous
  - Control returns to the CPU immediately

- CPU needs to synchronize before consuming the results

`cudaMemcpy()`	Blocks the CPU until the copy is complete Copy begins when all preceding CUDA calls have completed
`cudaMemcpyAsync()`	Asynchronous, does not block the CPU
`cudaDeviceSynchronize()`	Blocks the CPU until all preceding CUDA calls have completed

© NVIDIA 2013

Reporting Errors

- All CUDA API calls return an error code (`cudaError_t`)
  - Error in the API call itself

    OR
  - Error in an earlier asynchronous operation (e.g. kernel)

- Get the error code for the last error:

```
cudaError_t cudaGetLastError(void)
```

- Get a string to describe the error:

```
char *cudaGetErrorString(cudaError_t)
```

```
printf("%s\n", cudaGetErrorString(cudaGetLastError()));
```

© NVIDIA 2013

Device Management

- Application can query and select GPUs

```
cudaGetDeviceCount(int *count)
cudaSetDevice(int device)
cudaGetDevice(int *device)
cudaGetDeviceProperties(cudaDeviceProp *prop, int device)
```

- Multiple threads can share a device

- A single thread can manage multiple devices

  `cudaSetDevice(i)` to select current device

  `cudaMemcpy(…)` for peer-to-peer copies†

† requires OS and device support

Compute Capability

- The compute capability of a device describes its architecture, e.g.
  - Number of registers
  - Sizes of memories
  - Features & capabilities

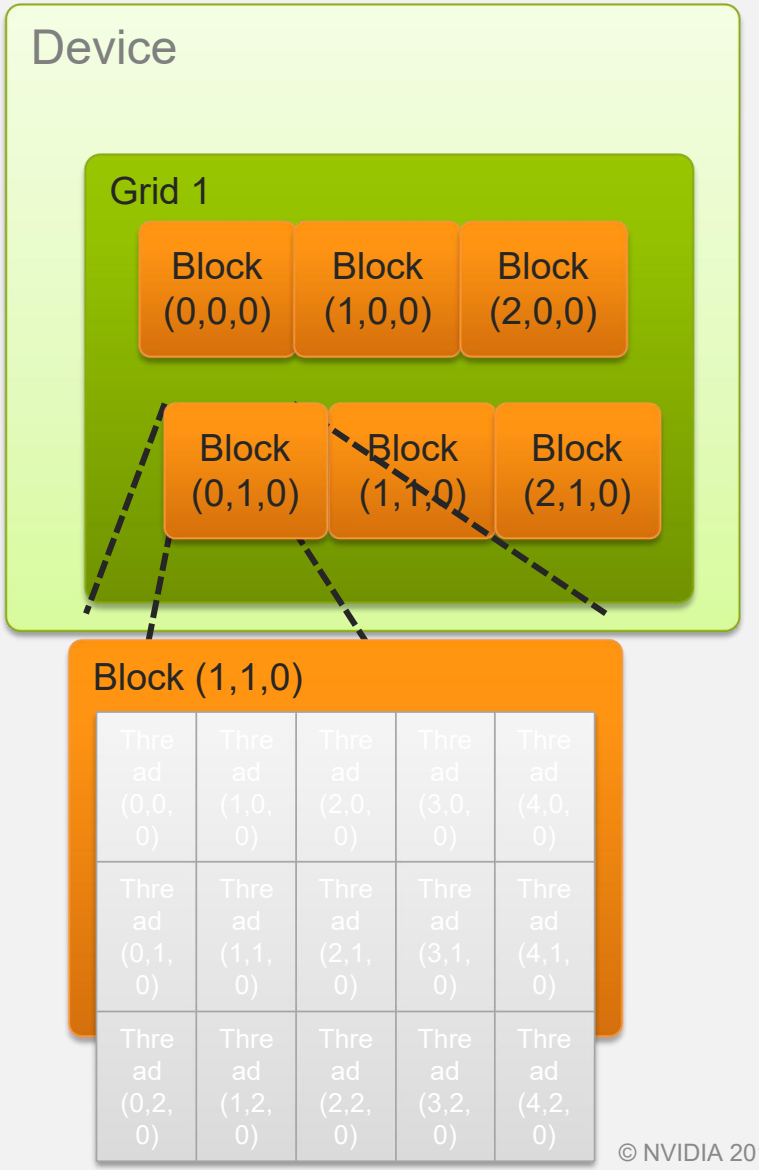
Compute Capability	Selected Features (see CUDA C Programming Guide for complete list)	Tesla models
1.0	Fundamental CUDA support	870
1.3	Double precision, improved memory accesses, atomics	10-series
2.0	Caches, fused multiply-add, 3D grids, surfaces, ECC, P2P, concurrent kernels/copies, function pointers, recursion	20-series

- The following presentations concentrate on Fermi devices
  - Compute Capability >= 2.0

© NVIDIA 2013

IDs and Dimensions

– A kernel is launched as a grid of blocks of threads
  - `blockIdx` and `threadIdx` are 3D
  - We showed only one dimension (`x`)

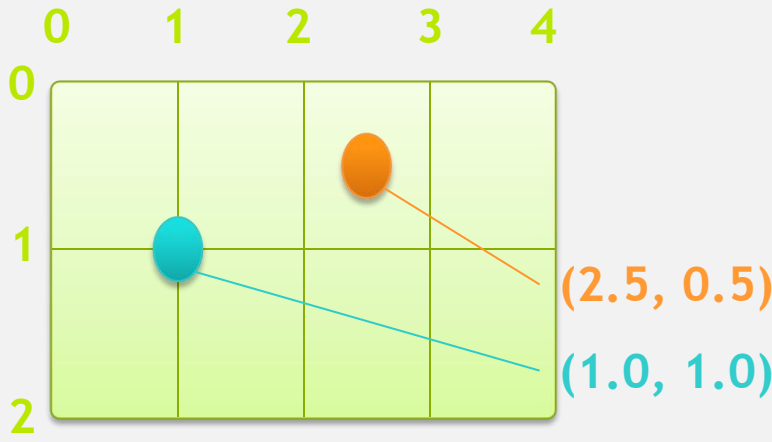


- Built-in variables:
  - `threadIdx`
  - `blockIdx`
  - `blockDim`
  - `gridDim`

© NVIDIA 2013

Textures

- Read-only object
  - Dedicated cache
- Dedicated filtering hardware
  (Linear, bilinear, trilinear)
- Addressable as 1D, 2D or 3D
- Out-of-bounds address handling
  (Wrap, clamp)



© NVIDIA 2013